

ReviewMind

AI-Native Code Review Platform

v0.1 - MVP Phase

MCP Protocol

Claude API

FastAPI

PostgreSQL

Redis

A2A Agents

GitHub API

1. Project Overview

Problem

Code review is one of the highest-leverage, highest-friction activities in software engineering. Reviewers must understand the intent of a change, spot security issues, enforce style conventions, and do all of this repeatedly across many PRs - often under time pressure. Most existing tools (linters, CI bots) operate in isolation and don't understand context.

Solution

ReviewMind is an AI-native code review co-pilot. It connects to GitHub, understands the semantic structure of a diff, runs security and style checks, and drafts high-quality review comments - all accessible from your own frontend via the Claude API and the Model Context Protocol (MCP).

Why This Architecture?

- MCP (Model Context Protocol) decouples the AI interface from the business logic - the same tools work in Claude Desktop, your own frontend, or any future MCP-compatible client.
- Claude API handles the reasoning: it decides which tools to call and in what order based on the user's natural-language input.
- FastAPI acts as both the MCP client (executing tool calls) and the HTTP API (webhooks, future SaaS endpoints).
- A2A (Agent-to-Agent) enables parallel sub-agents - security, style, and diff engines can run concurrently, reducing review time on large PRs.
- Redis + Postgres separation: Redis caches volatile GitHub API responses (short TTL), Postgres stores canonical audit data (findings, reviews, history).

Core User Workflow

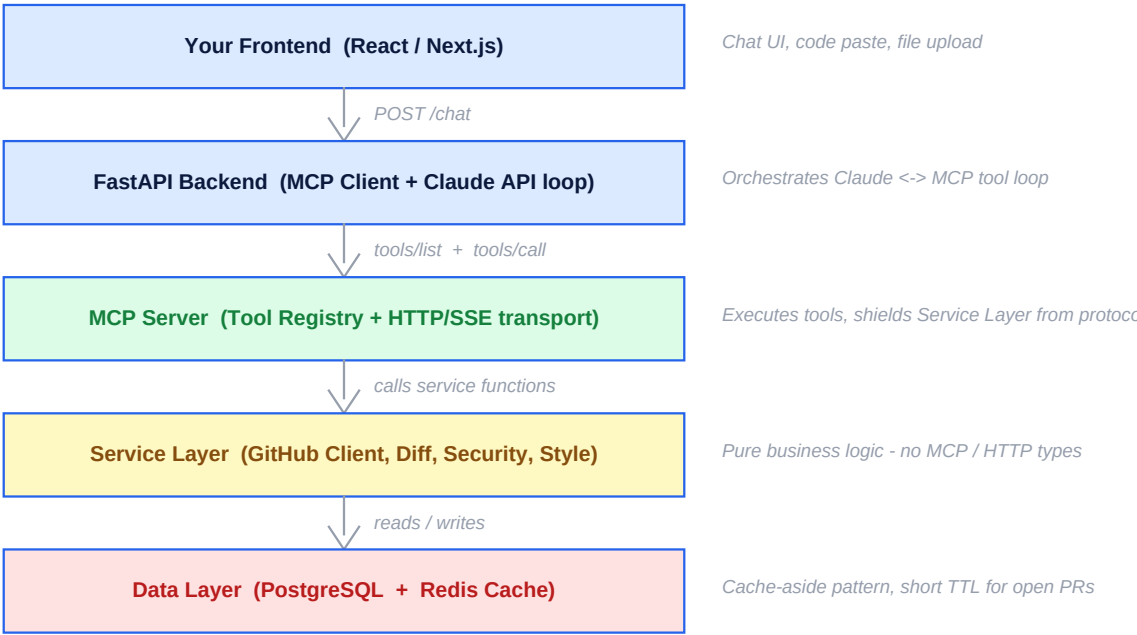
1. User opens your frontend and types: "Review PR #42 on abhishek/my-repo"
2. Frontend sends the message to FastAPI backend -> backend passes it to Claude API with tool definitions
3. Claude decides: call `get_pr_summary` -> `analyze_diff` -> `check_security_issues` + `check_style_violations` (parallel)
4. Backend executes each tool via MCP -> MCP server calls GitHub API, runs engines, returns findings
5. Claude synthesizes all findings into a structured natural-language review
6. User: "Post this as a review" -> Claude calls `post_review` -> comment appears on GitHub PR

Tech Stack

Component	Technology	Purpose
AI Brain	Claude API (claude-sonnet-4-6)	Decides which tools to call, synthesizes final review
Tool Protocol	MCP (HTTP/SSE transport)	Standard interface between Claude and ReviewMind tools
API Layer	FastAPI + Uvicorn	HTTP endpoints, webhook receiver, MCP client loop
Frontend	React / Next.js (planned)	Chat UI, code paste box, file upload, findings display
GitHub	GitHub REST API + httpx	Fetch PRs, diffs, blame, post review comments
Cache	Redis	Cache GitHub responses, avoid rate limits
Database	PostgreSQL + SQLAlchemy async	Store PRs, findings, review history
Migrations	Alembic	Schema versioning
Agent Protocol	A2A (planned)	Parallel sub-agents: security, style, diff

2. Architecture - System Layers

Five layers, each with one job. No layer knows about the internals of another - they communicate only through declared interfaces. This is the 'ports & adapters' pattern that makes every layer independently testable and replaceable.

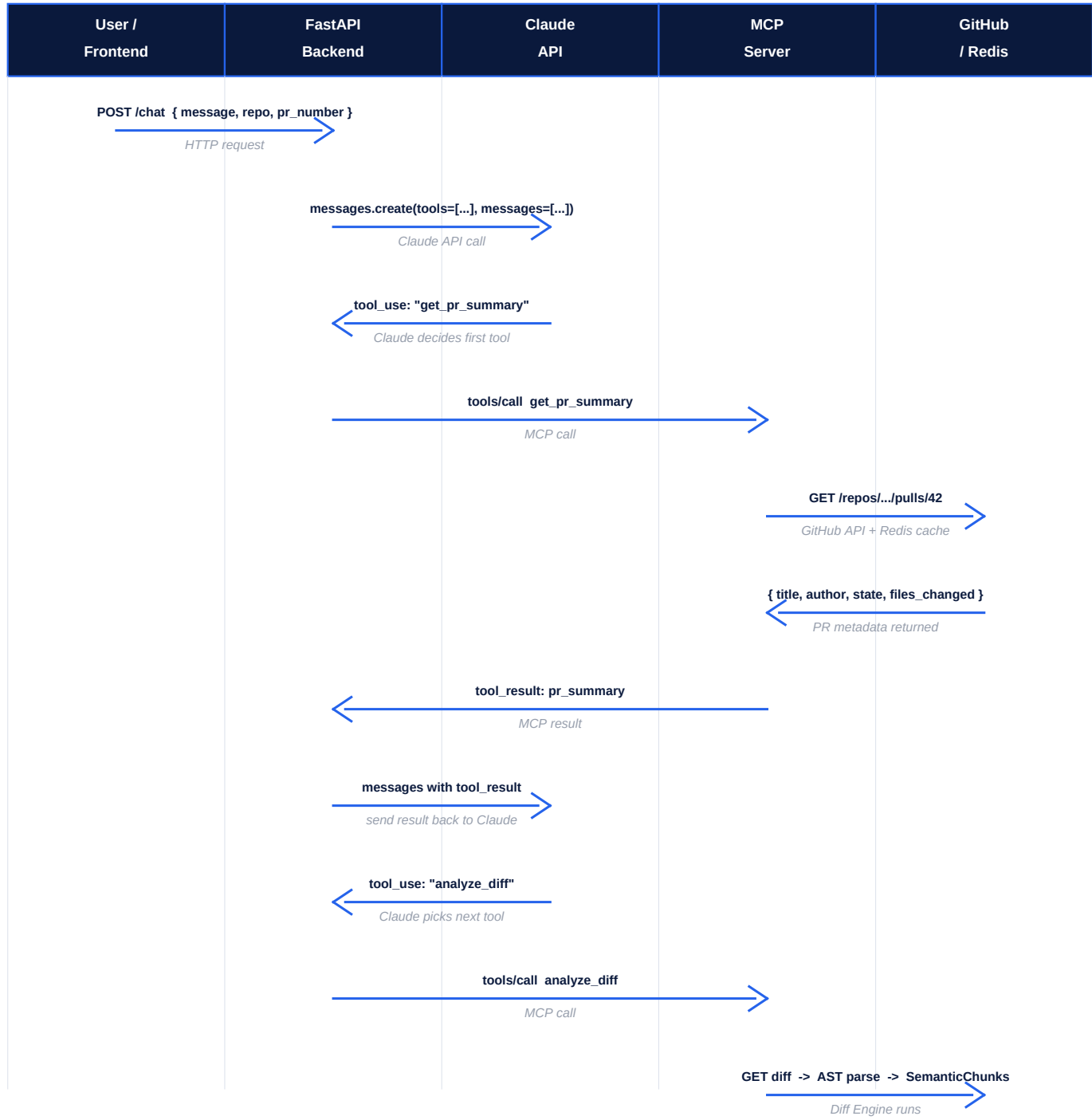


External Systems

System	Role
GitHub REST API	Source of truth for PRs, diffs, files, blame, reviews
Claude API	AI reasoning - decides tool call order, synthesizes review
OSV.dev (planned)	Live vulnerability database for dependency scanning

3. Flow: Reviewing a GitHub PR

Triggered when the user types something like "Review PR #42 on abhishek/my-repo". Claude decides the tool sequence autonomously - ReviewMind just executes what Claude asks for.



-> check_security_issues + check_style_violations called IN PARALLEL -> Claude synthesizes -> response to frontend

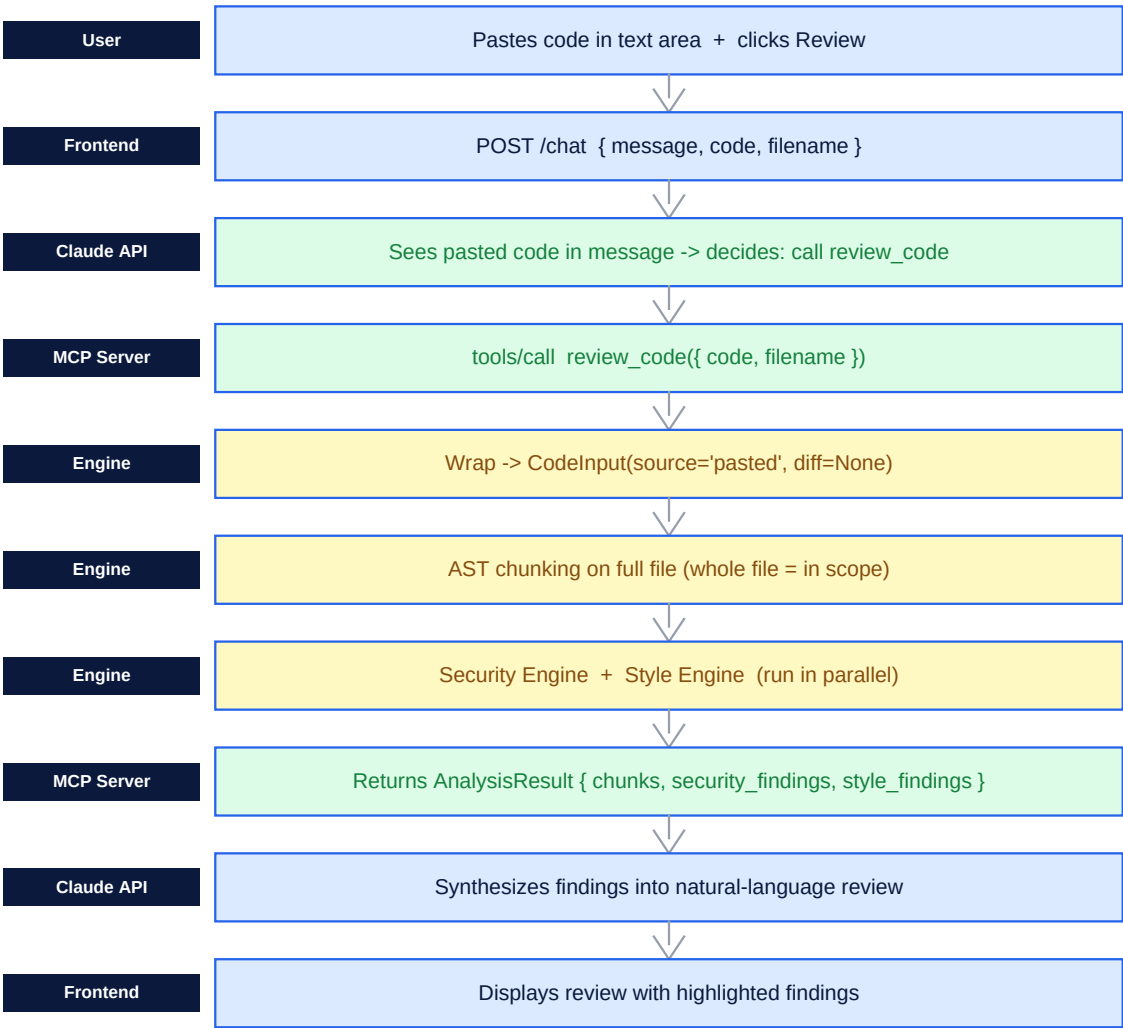
4. Flow: Reviewing Pasted Code

Triggered when the user pastes a code snippet directly into the frontend. No GitHub API calls - everything happens locally. One MCP tool call does the full analysis.

Comparison: PR Review vs Pasted Code

Aspect	PR Review	Pasted Code
Input source	GitHub PR number	Raw code string + optional filename
GitHub API calls	Yes (PR metadata, diff, files)	None
Redis caching	Yes - saves GitHub rate limits	None (stateless, one-off)
Postgres storage	Yes - PR, chunks, findings	None - ephemeral result
MCP tools called	4-6 tools per review	1 tool: review_code
Diff parsing	Yes - only changed lines	No - whole file in scope
Speed	Slower (network round-trips)	Fast (pure local analysis)
Use case	Team code review workflow	Quick one-off snippet check

Step-by-Step Flow



Key Insight: review_code is 100% stateless.
No DB write, no GitHub call, no cache. Wrap -> Chunk -> Scan -> Return. Fast and simple.

5b. Flow: Reviewing an Uploaded File

Users can drag-and-drop or select one or more files directly in the frontend. ReviewMind reads the file content in the browser, sends it to the backend, and runs the full security + style pipeline on each file. No GitHub needed - works completely offline.

Three Ways to Submit Code for Review

Aspect	GitHub PR	Paste Code	Upload File
Input method	PR number + repo name	Type/paste in text area	Drag-drop or file picker
GitHub API needed	Yes	No	No
Filename known	Yes (from diff)	Optional (user types it)	Yes (from file system)
Multiple files	Yes (all PR files)	No (one snippet)	Yes (batch upload)
Diff available	Yes	No	No
Results stored in DB	Yes	No	No (stateless)
MCP tool called	4-6 tools	review_code	review_file
Best for	Team PR workflow	Quick snippet check	Reviewing local files

Step-by-Step: Upload File Flow

Step 1	Frontend	User drops file(s) onto upload zone (or clicks file picker)
Step 2	Frontend	Browser reads content -> builds: [{filename, content}, ...]
Step 3	Frontend	POST /chat { message: "Review these files", files: [...] }
Step 4	Claude API	Reads file list in message -> decides: call review_file
Step 5	MCP Server	tools/call review_file({ files: [{filename, content}] })
Step 6	Engine	For each file: wrap -> CodeInput(source='uploaded', diff=None)
Step 7	Engine	Detect language from filename (.py / .ts / .go / .yaml)
Step 8	Engine	AST chunking per file (whole file in scope - no diff)
Step 9	Engine	Security Engine + Style Engine run in parallel per file
Step 10	MCP Server	Returns findings grouped by file: [{file, chunks, findings}]
Step 11	Claude API	Synthesizes per-file review + overall summary across files
Step 12	Frontend	Displays one findings tab per file + inline highlights

Supported File Types & Key Design Decisions

Supported File Types (MVP)	Key Design Decisions
.py - Python (full AST analysis)	No DB write - upload reviews are stateless
.js / .ts - JavaScript / TypeScript (regex)	Results per file, not one merged blob
.go - Go (regex-based for MVP)	diff=None -> whole file is in scope
.yaml/.json/.toml - Config files	Same engine as PR review - zero code duplication
.txt - Plain text (style rules only)	File size limit enforced in frontend (2 MB max)

review_file = review_code but for multiple files.
Same CodeInput -> Chunk -> Scan pipeline. Only the adapter layer differs.

5c. Flow: Review -> Suggest Fix -> Auth Gate -> Apply Fix

The core principle: READING is free and open, WRITING requires identity. A user can review any code without logging in. The moment they want to apply a fix to their code, ReviewMind triggers an auth gate. After login the exact fix they approved is applied - nothing else.

Auth Rule: What Requires Login?

Action	Auth Required?	Reason
detect_language, explain_code	No	Read-only - just understanding the code
review_code, review_file	No	Read-only - analysis produces no side effects
suggest_fix (per finding)	No	Read-only - just a suggestion, nothing applied yet
apply_fix (write the change)	YES	Mutates code - must know WHO made this change
post_review (write to GitHub)	YES	Mutates GitHub PR - requires GitHub identity

Complete Step-by-Step Flow

Phase	Auth	Actor	What Happens
REVIEW	No	Frontend	User pastes code or uploads file(s)
	No	MCP Server	detect_language -> identifies .py / .ts / .go
	No	MCP Server	explain_code -> plain-English summary of what the code does
	No	MCP Server	review_code / review_file -> Security + Style findings
	No	Claude API	Presents findings to user: 3 issues found (1 critical, 2 medium)
FIX	No	Claude API	suggest_fix for each finding -> shows diff of what would change
	No	Frontend	User reviews suggested fixes, clicks 'Apply Fix' on one
AUTH	YES	Backend	Auth check: is this user authenticated? -> NO
	YES	Frontend	Show login modal (email/password or GitHub OAuth)
	YES	Backend	User logs in -> JWT token issued -> session created
APPLY	YES	Backend	Auth check passes -> confirmed=True passed to apply_fix
	YES	MCP Server	apply_fix(finding_id, code, confirmed=True) executes
	YES	MCP Server	Returns patched code with only the approved fix applied
	YES	Frontend	Displays diff: original vs fixed. User can copy or download.

Key Design Rules for apply_fix

- One fix at a time - apply_fix takes a single finding_id, not a list. User must confirm each fix individually.
- confirmed flag is required - Claude always passes confirmed=True only AFTER the user explicitly clicks 'Apply'. No silent auto-apply.
- apply_fix never calls GitHub - it returns patched code to the frontend. The user decides what to do with it (copy, download, or paste back).
- The fix is scoped exactly - only the lines from the finding are changed. Nothing else in the file is touched.
- Audit trail - every apply_fix call is logged with user_id + finding_id + timestamp even though the result is not stored in DB.

6. MCP Tools Registry (19 Tools)

All tools are registered on the MCP server. Claude reads their names and descriptions at startup (tools/list) and decides which to call based on the user's message. Tools marked Auth=Yes require the user to be authenticated before the backend executes them.

Tool Name	Input	Purpose	Auth	Status
GitHub PR Tools (no auth required to read)				
get_pr_summary	owner, repo, pr_number	PR metadata: title, author, state, files changed	No	Planned
get_pr_context	owner, repo, pr_number	Related files, commit history, linked issues	No	Planned
analyze_diff	owner, repo, pr_number	Parse diff into semantic chunks (functions/classes)	No	Planned
check_security_issues	owner, repo, pr_number	Scan for secrets, SQL injection, unsafe patterns	No	Planned
check_style_violations	owner, repo, pr_number	Enforce reviewmind.yaml project conventions	No	Planned
check_dependency_changes	owner, repo, pr_number	Cross-check added packages against OSV vuln DB	No	Planned
get_pr_diff_file	owner, repo, pr_number, path	Full file content for deep context on a specific file	No	Planned
suggest_reviewers	owner, repo, pr_number	Blame-based reviewer suggestions + CODEOWNERS	No	Planned
generate_review_comment	owner, repo, pr_number	Assemble findings into structured draft review	No	Planned
post_review	owner, repo, pr_number, body ...	Post review to GitHub (APPROVE / REQUEST_CHANGES)	Yes	Planned
Paste + Upload Tools (review = no auth apply = auth required)				
detect_language	code, filename?	Detect programming language - runs before all analysis	No	Planned
explain_code	code, filename	Plain-English explanation of what the code does	No	Planned
review_code	code: str, filename?: str	Full security+style analysis on pasted code snippet	No	Planned
review_file	files: [{filename, content}]	Analyse one or more uploaded files, results per file	No	Planned
suggest_fix	finding_id, code, language	Suggest a concrete fix for one specific finding	No	Planned
apply_fix	finding_id, code, confirmed:bool	Apply a suggested fix - requires user confirmation	Yes	Planned
Utility Tools				
explain_finding	rule_id	Detailed explanation + examples for a specific rule	No	Planned
get_analysis_history	owner, repo	Past findings for this repo - track regressions	No	Planned
ping	-	Health-check: verify MCP server is reachable	No	Done [DONE]

How Claude Uses These Tools

Claude never calls tools in a hardcoded sequence - it assembles the right sequence dynamically based on user intent. The auth gate only blocks execution, not discovery: Claude can still describe what apply_fix would do before the user logs in.

- "What does this code do?" -> detect_language -> explain_code
- "Review my pasted code" -> detect_language -> review_code -> suggest_fix (per finding)
- "Apply the SQL fix" -> auth check -> apply_fix (after user confirms)
- "Review this file" -> detect_language -> review_file -> suggest_fix (per finding)
- "Just check for secrets on this PR" -> check_security_issues only
- "Post the review" -> auth check -> post_review

6. Analysis Engines

Security Rules (SEC001 - SEC005)

Heuristic, pattern-based static analysis - fast, explainable, PR-scoped. Not a replacement for Semgrep/CodeQL, but a high-signal first pass.

Rule ID	Name	Severity	Detection Logic
SEC001	Hardcoded Secret	High	Regex scan added lines for API keys, tokens, private key headers
SEC002	SQL Injection	Critical	f-string / .format() / % used in DB execute() calls
SEC003	Insecure Dependency	Medium	Added package versions cross-checked against OSV.dev snapshot
SEC004	Missing Input Validation	Low	Request handler params used in file/DB/subprocess without guard
SEC005	Unsafe Deserialization	High	pickle.load, yaml.load (no SafeLoader), eval, exec, shell=True

Style Rules (STY001 - STY006)

Loaded from reviewmind.yaml in the repo root - every rule is toggleable per project. Severity is configurable; defaults shown below.

Rule ID	Name	Default Severity	Trigger Condition
STY001	Max Function Length	Configurable	Function exceeds max_function_length lines in reviewmind.yaml
STY002	Missing Docstring	Configurable	Public function/class has no docstring
STY003	Naming Convention	Configurable	Identifier doesn't match snake_case / PascalCase regex
STY004	Forbidden Import	Configurable	Import matches a forbidden pattern (e.g. 'from x import *')
STY005	Max Line Length	Configurable	Line exceeds max_line_length characters
STY006	TODO without ticket	Info	TODO/FIXME comment has no issue reference

Diff Engine Pipeline

- Raw unified diff -> parse files + hunks (unidiff library)
- Fetch full post-change file content from GitHub Contents API
- AST parse with Python ast module -> map hunks to functions / classes
- Merge hunks in same function into one SemanticChunk
- Classify: new_feature / bug_fix / refactor / test_change / config_change / dependency_change
- Estimate cyclomatic complexity per function; flag complexity_delta > threshold
- Extract called functions (ast.Call nodes) for intra-PR dependency detection
- Return AnalysisResult { chunks: SemanticChunk[] } -> consumed by Security + Style engines

7. Database Schema

Five tables, designed around the 'repositories are the anchor' pattern. Redis caches derived/volatile data; Postgres stores canonical records.

Table	Key Columns	Purpose
repositories	id, owner, name, default_branch, config_yaml	Anchor entity - every repo ReviewMind has seen
pull_requests	id, repository_id, pr_number, title, author, state, head_sha	Cached PR metadata + invalidation key (head_sha)
diff_chunks	id, pull_request_id, file_path, chunk_type, content	Semantic chunks from Diff Engine - reused by scanners
findings	id, pull_request_id, category, rule_id, severity, file, line	All security + style findings (audit trail)
reviews	id, pull_request_id, github_review_id, event, html_url	Every review ReviewMind has posted to GitHub

Entity Relationships

- repositories 1 -> many pull_requests (one repo has many PRs)
- pull_requests 1 -> many diff_chunks (one PR produces many semantic chunks)
- pull_requests 1 -> many findings (one PR has many security/style findings)
- pull_requests 1 -> many reviews (one PR may have multiple posted reviews)
- diff_chunks 1 -> many findings (optional) (a finding may reference a specific chunk)

Redis Cache Keys & TTL Strategy

Key Pattern	Data	TTL
pr:{owner}:{repo}:{n}:summary	PR metadata	5 min (open) / 24h (closed)
pr:{owner}:{repo}:{n}:diff	Raw unified diff	1h (open) / 7d (closed)
pr:{owner}:{repo}:{n}:files	Files changed list	Same as diff
pr:{owner}:{repo}:{n}:analysis	Full AnalysisResult	Invalidated on new head_sha
repo:{owner}:{repo}:config	reviewmind.yaml	1h (rarely changes)
github:etag:{endpoint_hash}	Last ETag header	No TTL (overwritten on update)

Indexing Strategy

- repositories: UNIQUE (owner, name)
- pull_requests: UNIQUE (repository_id, pr_number) | INDEX on head_sha (cache invalidation)
- diff_chunks: INDEX on (pull_request_id, file_path)
- findings: INDEX on (pull_request_id, category) | INDEX on rule_id (analytics)
- reviews: INDEX on pull_request_id

8. Build Status & Roadmap

What's Built vs Pending

Item	Status	Notes
Docker Compose (Postgres + Redis)	Done [DONE]	docker compose up starts both services
Core config + structured logging	Done [DONE]	core/config.py, core/logging.py
FastAPI app + /health endpoint	Done [DONE]	Checks DB + Redis connectivity
SQLAlchemy async models + Alembic	Done [DONE]	5 tables designed, migration created
Redis cache client	Done [DONE]	core/cache.py wired to FastAPI
MCP server (ping tool)	Done [DONE]	mcp_server.py - placeholder tool working
GitHub HTTP client	Pending	Phase 2 - httpx client with auth + ETags
Redis cache-aside for GitHub calls	Pending	Phase 2 - TTL strategy per Section 10
Semantic Diff Engine	Pending	Phase 3 - unidiff + AST chunking
Security Analysis Engine	Pending	Phase 4 - 5 rules (secrets, SQLi, etc.)
Style Analysis Engine	Pending	Phase 5 - reviewmind.yaml rule engine
All 16 MCP tools wired	Pending	Phase 6 - tools call Service Layer
Claude API tool loop in FastAPI	Pending	Phase 6 - /chat endpoint
Frontend (React/Next.js)	Pending	Phase 7 - chat UI + code paste + upload
A2A parallel sub-agents	Pending	Phase 8 - security/style agents in parallel

Phase Roadmap

Phase	Name	Status	Deliverables
Phase 1	Foundation	Done [DONE]	Docker, DB, Redis, FastAPI, MCP skeleton
Phase 2	GitHub Client	Next	httpx client, rate limiting, Redis caching, pagination
Phase 3	Diff Engine	Upcoming	unidiff parsing, AST chunking, classification, complexity
Phase 4	Security Engine	Upcoming	5 security rules, rule registry pattern
Phase 5	Style Engine	Upcoming	reviewmind.yaml, built-in style rules
Phase 6	MCP + Claude API	Upcoming	All 16 tools wired, /chat endpoint with tool loop
Phase 7	Frontend	Upcoming	React/Next.js chat UI, paste box, file upload
Phase 8	A2A Agents	Future	Parallel sub-agents for security/style/diff
Month 2	GitHub App + PyPI	Future	OAuth, GitHub App install, pip install reviewmind
Month 3	SaaS Platform	Future	Multi-tenant, billing, web dashboard, Redis Cluster

To update this PDF - edit scripts/generate_pdf.py and re-run:
python scripts/generate_pdf.py